

1、讲一讲Redis缓存的三大问题！（击穿、穿透、雪崩）

以下笔记来自Redis体系课，面试突击课只是抽取其中一些快速讲解：

缓存穿透、击穿、雪崩

缓存穿透

是指查询一个根本不存在的数据库，缓存层和存储层都不会命中，于是这个请求就可以随意访问数据库，这个就是缓存穿透，缓存穿透将导致不存在的数据每次请求都要到存储层去查询，失去了缓存保护后端存储的意义。

缓存穿透问题可能会使后端存储负载加大，由于很多后端存储不具备高并发性，甚至可能造成后端存储宕掉。通常可以在程序中分别统计总调用数、缓存层命中数、存储层命中数，如果发现大量存储层空命中，可能就是出现了缓存穿透问题。

造成缓存穿透的基本原因有两个。

第一，自身业务代码或者数据出现问题，比如，我们数据库的id都是1开始自增上去的，如发起为id值为-1的数据或id为特别大不存在的数据。如果不对参数做校验，数据库id都是大于0的，我一直用小于0的参数去请求你，每次都能绕开Redis直接打到数据库，数据库也查不到，每次都这样，并发高点就容易崩掉了。

第二，一些恶意攻击、爬虫等造成大量空命中。下面我们来看一下如何解决缓存穿透问题。

1.缓存空对象

当存储层不命中，到数据库查发现也没有命中，那么仍然将空对象保留到缓存层中，之后再访问这个数据将会从缓存中获取，这样就保护了后端数据源。

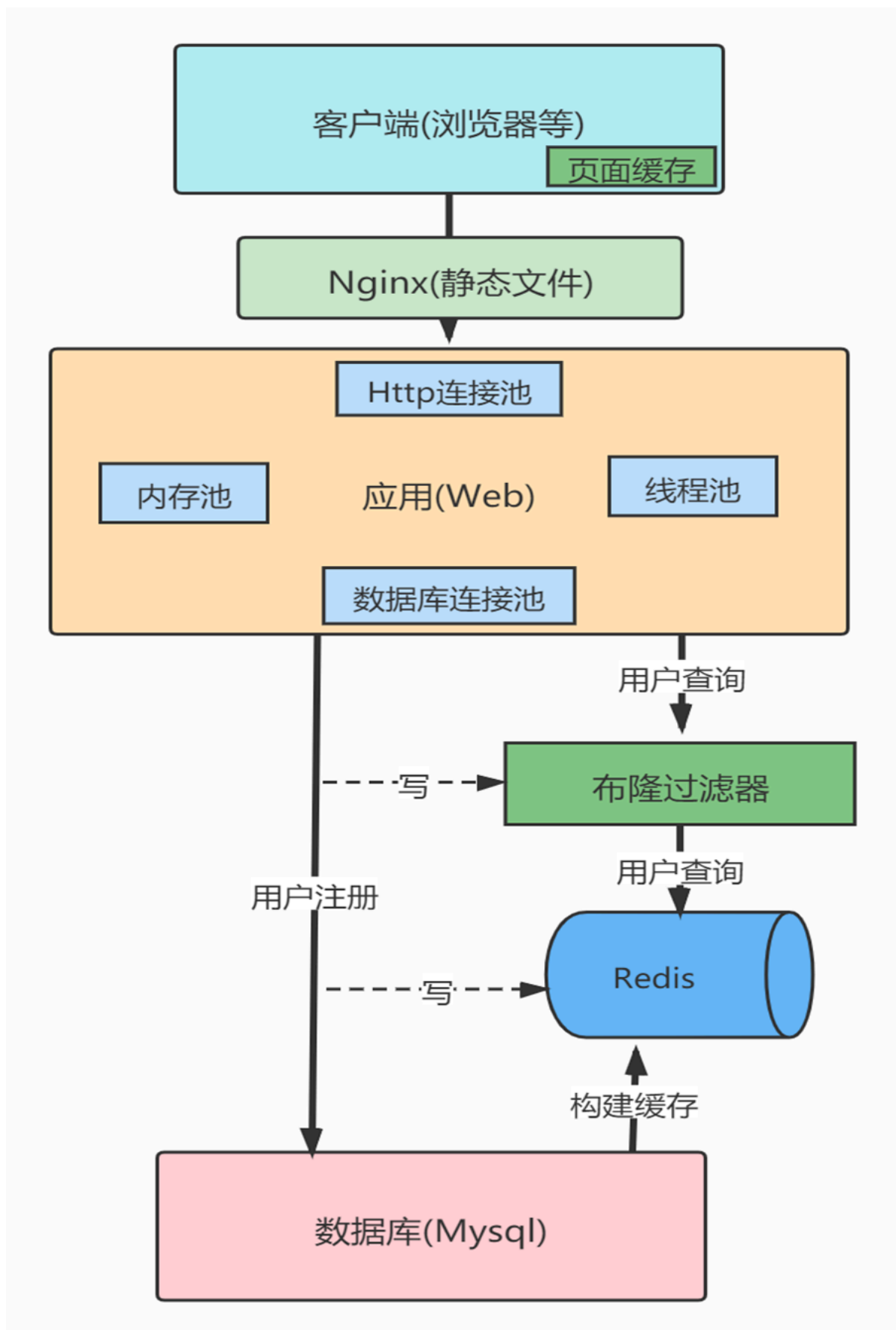
缓存空对象会有两个问题：

第一，空值做了缓存，意味着缓存层中存了更多的键，需要更多的内存空间(如果是攻击，问题更严重)，比较有效的方法是针对这类数据设置一个较短的过期时间，让其自动剔除。

第二，缓存层和存储层的数据会有一段时间窗口的不一致，可能会对业务有一定影响。例如过期时间设置为5分钟，如果此时存储层添加了这个数据，那此段时间就会出现缓存层和存储层数据的不一致，此时可以利用前面所说的数据一致性方案处理。

2.布隆过滤器拦截

在访问缓存层和存储层之前，将存在的key用布隆过滤器提前保存起来，做第一层拦截。例如：一个推荐系统有4亿个用户id，每小时算法工程师会根据每个用户之前历史行为计算出推荐数据放到存储层中，但是最新的用户由于没有历史行为，就会发生缓存穿透的行为，为此可以将所有推荐数据的用户做成布隆过滤器。如果布隆过滤器认为该用户id不存在，那么就不会访问存储层，在一定程度上保护了存储层。



缓存击穿

缓存击穿是指一个Key非常热点，在不停的扛着大并发，大并发集中对这一个点进行访问，当这个Key在失效的瞬间，持续的大并发就穿破缓存，直接请求数据库，就像在一个完好无损的桶上凿开了一个洞。

缓存击穿的话，设置热点数据永远不过期。或者加上互斥锁就能搞定了。

使用互斥锁(mutex key)

业界比较常用的做法，是使用mutex。简单地来说，就是在缓存失效的时候（判断拿出来的值为空），不是立即去load db，而是先使用缓存工具的某些带成功操作返回值的操作（比如Redis的SETNX或者Memcache的ADD）去set一个mutex key，当操作返回成功时，再进行load db的操作并回设缓存；否则，就重试整个get缓存的方法。

伪代码如下图：

```
public String get(key){
    String value = redis.get(key);
    if(value == null) { //缓存过期
        //设置 3min的超时，防止
        if(redis.setnx(key_mutex,1 , 3*60) == 1) { //代表设置成功
            value = db.get(key);
            redis.set(key,value,expire_time)
            redis.del(key_mutex);
        }else{
            sleep(50);
            get(key); //重试
        }
    }else{
        return value; //缓存没过期就返回
    }
}
```

永远不过期

这里的“永远不过期”包含两层意思：

- (1) 从redis上看，确实没有设置过期时间，这就保证了，不会出现热点key过期问题，也就是“物理”不过期。
- (2) 从功能上看，如果不过期，那不就成了静态的吗？所以我们把过期时间存在key对应的value里，如果发现要过期了，通过一个后台的异步线程进行缓存的构建，也就是“逻辑”过期

从实战看，这种方法对于性能非常友好，唯一不足的就是构建缓存时候，其余线程(非构建缓存的线程)可能访问的是老数据，但是对于一般的互联网功能来说这个还是可以忍受。

缓存雪崩

缓存雪崩:由于缓存层承载着大量请求,有效地保护了存储层,但是如果缓存层由于某些原因不能提供服务,比如同一时间缓存数据大面积失效,那一瞬间Redis跟没有一样,于是所有的请求都会达到存储层,存储层的调用量会暴增,造成存储层也会级联宕机的情况。

缓存雪崩的英文原意是stampeding herd(奔逃的野牛),指的是缓存层宕掉后,流量会像奔逃的野牛一样,打向后端存储。

预防和解决缓存雪崩问题,可以从以下三个方面进行着手。

- 1) 保证缓存层服务高可用性。和飞机都有多个引擎一样,如果缓存层设计成高可用的,即使个别节点、个别机器、甚至是机房宕掉,依然可以提供服务,例如前面介绍过的Redis

Sentinel和 Redis Cluster都实现了高可用。

- 2) 依赖隔离组件为后端限流并降级。无论是缓存层还是存储层都会有出错的概率，可以将它们视同为资源。作为并发量较大的系统，假如有一个资源不可用，可能会造成线程全部阻塞(hang)在这个资源上，造成整个系统不可用。
- 3) 提前演练。在项目上线前，演练缓存层宕掉后，应用以及后端的负载情况以及可能出现的问题,在此基础上做一些预案设定。
- 4) 将缓存失效时间分散开，比如我们可以在原有的失效时间基础上增加一个随机值，比如1-5分钟随机，这样每一个缓存的过期时间的重复率就会降低，就很难引发集体失效的事件。

2、讲一讲Redis缓存的BigKey和热点Key！

以下笔记来自Redis体系课，面试突击课只是抽取其中一些快速讲解：

BigKey

什么是bigkey

bigkey是指key对应的value所占的内存空间比较大，例如一个字符串类型的value可以最大存到512MB，一个列表类型的value最多可以存储23-1个元素。

如果按照数据结构来细分的话，一般分为字符串类型bigkey和非字符串类型bigkey。

字符串类型：体现在单个value值很大，一般认为超过10KB就是bigkey，但这个值和具体的OPS相关。

非字符串类型：哈希、列表、集合、有序集合,体现在元素个数过多。

bigkey无论是空间复杂度和时间复杂度都不太友好，下面我们将介绍它的危害。

bigkey的危害

bigkey的危害体现在三个方面：

- 1、内存空间不均匀.(平衡):例如在Redis Cluster中，bigkey 会造成节点的内存空间使用不均匀。
- 2、超时阻塞:由于Redis单线程的特性，操作bigkey比较耗时，也就意味着阻塞Redis可能性增大。
- 3、网络拥塞:每次获取bigkey产生的网络流量较大

假设一个bigkey为1MB，每秒访问量为1000，那么每秒产生1000MB 的流量,对于普通的千兆网卡(按照字节算是128MB/s)的服务器来说简直是灭顶之灾，而且一般服务器会采用单机多实例的方式来部署,也就是说一个bigkey可能会对其他实例造成影响,其后果不堪设想。

bigkey的存在并不是完全致命的：

如果这个bigkey存在但是几乎不被访问,那么只有内存空间不均匀的问题存在,相对于另外两个问题没有那么重要紧急,但是如果bigkey是一个热点key(频繁访问)，那么其带来的危害不可想象,所以在实际开发和运维时一定要密切关注bigkey的存在。

发现bigkey

redis-cli --bigkeys可以命令统计bigkey的分布。


```
127.0.0.1:6379> scan 0
1) "9"
2) 1) "laoma10"
   2) "laoma2"
   3) "lijin4"
   4) "bigkey"
   5) "laoma"
   6) "lijin2"
   7) "laoma5"
   8) "lijin"
   9) "laoma1"
  10) "laoma7"
  11) "laoma9"
127.0.0.1:6379> scan 9
1) "0"
2) 1) "lijin3"
   2) "laoma8"
   3) "lijin1"
   4) "laoma3"
   5) "laoma6"
   6) "lijin5"
   7) "laoma4"
   8) "laoma11"
```

可以看到，第一次执行scan 0，返回结果分为两个部分：

第一个部分9就是下次scan需要的cursor

第二个部分是10个键。接下来继续

直到得到结果cursor变为0，说明所有的键已经被遍历过了。

除了scan 以外，Redis提供了面向哈希类型、集合类型、有序集合的扫描遍历命令，解决诸如hgetall、smembers、zrange可能产生的阻塞问题，对应的命令分别是hscan、sscan、zscan，它们的用法和scan基本类似，请自行参考Redis官网。

```
127.0.0.1:6379> sscan lijins 0
1) "0"
2) 1) "hello"
   2) "cs"
   3) "18"
```

渐进式遍历可以有效的解决keys命令可能产生的阻塞问题，但是scan并非完美无瑕，如果在scan 的过程中如果有键的变化(增加、删除、修改)，那么遍历效果可能会碰到如下问题:新增的键可能没有遍历到，遍历出了重复的键等情况，也就是说scan并不能保证完整的遍历出来所有的键，这些是我们在开发时需要考虑的。

如果键值个数比较多，scan + debug object会比较慢，可以利用Pipeline机制完成。对于元素个数较多的数据结构，debug object执行速度比较慢，存在阻塞Redis的可能，所以如果有从节点,可以考虑在从节点上执行。

解决bigkey

主要思路为拆分，对 big key 存储的数据（big value）进行拆分，变成value1，value2... valueN等等。

例如big value 是个大json 通过 mset 的方式，将这个 key 的内容打散到各个实例中，或者一个hash，每个field代表一个具体属性，通过hget、hmget获取部分value，hset、hmset来更新部分属性。

例如big value 是个大list，可以拆成将list拆成。= list_1，list_2, list3, ...listN

其他数据类型同理。

热点Key

在Redis中，访问频率高的key称为热点key。

产生原因和危害

原因

热点问题产生的原因大致有以下两种：

用户消费的数据远大于生产的数据（热卖商品、热点新闻、热点评论、明星直播）。

在日常工作生活中一些突发的事件，例如：双十一期间某些热门商品的降价促销，当这其中的某一件商品被数万次点击浏览或者购买时，会形成一个较大的需求量，这种情况下就会造成热点问题。同理，被大量刊发、浏览的热点新闻、热点评论、明星直播等，这些典型的读多写少的场景也会产生热点问题。

请求分片集中，超过单Server的性能极限。在服务端读数据进行访问时，往往会对数据进行分片切分，此过程中会在某一主机Server上对相应的Key进行访问，当访问超过Server极限时，就会导致热点Key问题的产生。

危害

- 1、流量集中，达到物理网卡上限。
- 2、请求过多，缓存分片服务被打垮。
- 3、DB击穿，引起业务雪崩。

发现热点key

预估发现

针对业务提前预估出访问频繁的热点key，例如秒杀商品业务中，秒杀的商品都是热点key。

当然并非所有的业务都容易预估出热点key，可能出现漏掉或者预估错误的情况。

客户端发现

客户端其实是距离key"最近"的地方，因为Redis命令就是从客户端发出的，以Jedis为例，可以在核心命令入口，使用这个Google Guava中的AtomicLongMap进行记录，如下所示。

使用客户端进行热点key的统计非常容易实现，但是同时问题也非常多：

- (1) 无法预知key的个数，存在内存泄露的危险。
- (2) 对于客户端代码有侵入，各个语言的客户端都需要维护此逻辑，维护成本较高。
- (3) 规模化汇总实现比较复杂。

Redis发现

monitor命令

monitor命令可以监控到Redis执行的所有命令，利用monitor的结果就可以统计出一段时间内的热点key排行榜，命令排行榜，客户端分布等数据。

```
127.0.0.1:6379> monitor
OK
1667896001.360067 [0 127.0.0.1:11020] "COMMAND"
1667896008.307495 [0 127.0.0.1:11020] "set" "55" "666"
127.0.0.1:6379> set 55 666
OK
127.0.0.1:6379>
```

Facebook开源的redis-faina正是利用上述原理使用Python语言实现的，例如下面获取最近10万条命令的热点key、热点命令、耗时分布等数据。为了减少网络开销以及加快输出缓冲区的消费速度，monitor尽可能在本机执行。

此种方法会有两个问题：

- 1、monitor命令在高并发条件下，内存暴增同时会影响Redis的性能，所以此种方法适合在短时间内使用。
- 2、只能统计一个Redis节点的热点key，对于Redis集群需要进行汇总统计。

可以参考的框架：Facebook开源的redis-faina正是利用上述原理使用Python语言实现的

hotkeys

Redis在4.0.3中为redis-cli提供了--hotkeys，用于找到热点key。

```
[root@centosvm bin]# ./redis-cli --hotkeys
# Scanning the entire keyspace to find hot keys as well as
# average sizes per key type. You can use -i 0.1 to sleep 0.1 sec
# per 100 SCAN commands (not usually needed).
Error: ERR An LRU maxmemory policy is not selected, access frequency not tracked. Please note that when switching between policies at runtime take some time to adjust.
```

如果有错误，需要先把内存逐出策略设置为allkeys-lfu或者volatile-lfu，否则会返回错误。

```
[root@centosvm bin]# ./redis-cli
127.0.0.1:6379> config set maxmemory-policy volatile-lfu
OK
127.0.0.1:6379> exit
```

```
[root@centosvm bin]# ./redis-cli --hotkeys
# Scanning the entire keyspace to find hot keys as well as
# average sizes per key type. You can use -i 0.1 to sleep 0.1 sec
# per 100 SCAN commands (not usually needed).

[00.00%] Hot key '"lijin"' found so far with counter 8

----- summary -----

Sampled 6 keys in the keyspace!
hot key found with counter: 8 keyname: "lijin"
```

但是如果键值较多，执行较慢，和热点的概念的有点背道而驰，同时热度定义的不够准确。

抓取TCP包发现

Redis客户端使用TCP协议与服务端进行交互，通信协议采用的是RESP。如果站在机器的角度，可以通过对机器上所有Redis端口的TCP数据包进行抓取完成热点key的统计

此种方法对于Redis客户端和服务端来说毫无侵入，是比较完美的方案，但是依然存在3个问题：

(1) 需要一定的开发成本

(2) 对于高流量的机器抓包，对机器网络可能会有干扰，同时抓包时候会有丢包的可能性。

(3) 维护成本过高。

对于成本问题，有一些开源方案实现了该功能，例如ELK(ElasticSearch Logstash Kibana)体系下的packetbeat[2] 插件，可以实现对Redis、MySQL等众多主流服务的数据包抓取、分析、报表展示优

解决热点key

发现热点key之后，需要对热点key进行处理。

使用二级缓存

可以使用 guava-cache或hcache，发现热点key之后，将这些热点key加载到VM中作为本地缓存。访问这些key时直接从本地缓存获取即可，不会直接访问到redis层了，有效的保护了缓存服务器。

key分散

将热点key分散为多个子key，然后存储到缓存集群的不同机器上，这些子key对应的value都和热点key是一样的。当通过热点key去查询数据时，通过某种hash算法随机选择一个子key，然后再去访问缓存机器，将热点分散到了多个子key上。

3、讲一讲Redis缓存的主从复制

以下笔记来自Redis体系课，面试突击课只是抽取其中一些快速讲解：

复制

在分布式系统中为了解决单点问题，通常会把数据复制多个副本部署到其他机器，满足故障恢复和负载均衡等需求。Redis也是如此，它为我们提供了复制功能，实现了相同数据的多个Redis 副本。复制功能是高可用Redis的基础，后面章节的哨兵和集群都是在复制的基础上实现高可用的。

默认情况下，Redis都是主节点。每个从节点只能有一个主节点，而主节点可以同时具有多个从节点。复制的数据流是单向的，只能由主节点复制到从节点。

复制的拓扑结构

Redis 的复制拓扑结构可以支持单层或多层复制关系，根据拓扑复杂性可以分为以下三种：一主一从、一主多从、树状主从结构,下面分别介绍。

一主一从结构

一主一从结构是最简单的复制拓扑结构，用于主节点出现宕机时从节点提供故障转移支持。

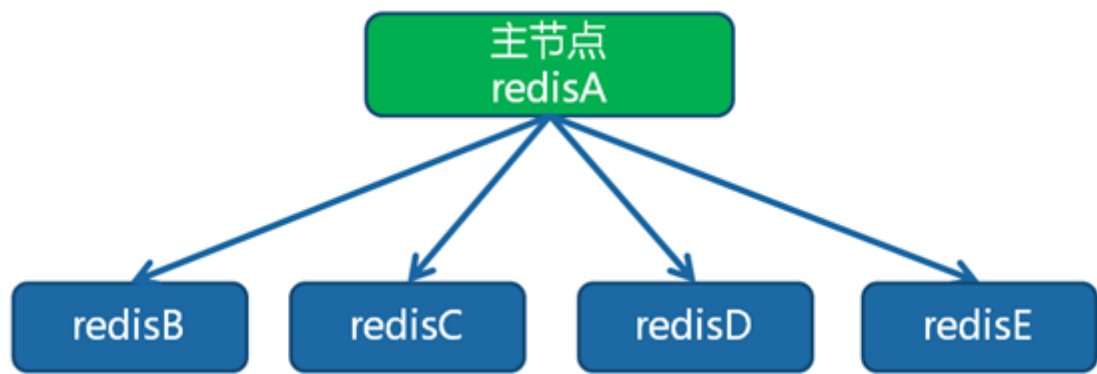


当应用写命令并发量较高且需要持久化时,可以只在从节点上开启AOF,这样既保证数据安全性同时也避免了持久化对主节点的性能干扰。但需要注意的是，当主节点关闭持久化功能时，如果主节点脱机要避免自动重启操作。

因为主节点之前没有开启持久化功能自动重启后数据集为空，这时从节点如果继续复制主节点会导致从节点数据也被清空的情况,丧失了持久化的意义。安全的做法是在从节点上执行slaveof no one断开与主节点的复制关系，再重启主节点从而避免这一问题。

一主多从结构

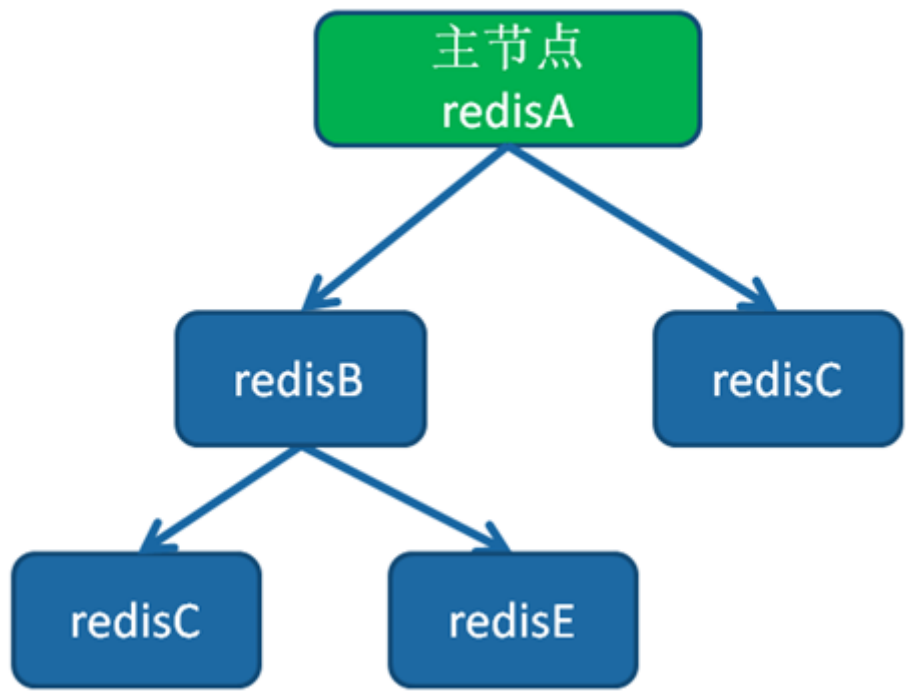
一主多从结构(又称为星形拓扑结构) 使得应用端可以利用多个从节点实现读写分离。



对于读占比较大的场景，可以把读命令发送到从节点来分担主节点压力。同时在日常开发中如果需要执行一些比较耗时的读命令，如:keys、sort等，可以在其中一台从节点上执行，防止慢查询对主节点造成阻塞从而影响线上服务的稳定性。对于写并发量较高的场景,多个从节点会导致主节点写命令的多次发送从而过度消耗网络带宽，同时也加重了主节点的负载影响服务稳定性。

树状主从结构

树状主从结构(又称为树状拓扑结构) 使得从节点不但可以复制主节点数据，同时可以作为其他从节点的主节点继续向下层复制。通过引入复制中间层，可以有效降低主节点负载和需要传送给从节点的数据量。



数据写入节点A后会同步到B和C节点,B节点再把数据同步到D和E节点,数据实现了一层一层的向下复制。当主节点需要挂载多个从节点时为了避免对主节点的性能干扰,可以采用树状主从结构降低主节点压力

4、讲一讲Redis缓存的哨兵机制

哨兵Redis Sentinel

Redis 的主从复制模式下，一旦主节点由于故障不能提供服务，需要人工将从节点晋升为主节点，同时还需要通知应用方更新主节点地址，对于很多应用场景这种故障处理的方式是无法接受的。

Redis 从 2.8开始正式提供了Redis Sentinel(哨兵) 架构来解决这个问题。

主从复制的问题

Redis 的主从复制模式可以将主节点的数据改变同步给从节点，这样从节点就可以起到两个作用

第一，作为主节点的一个备份，一旦主节点出了故障不可达的情况，从节点可以作为后备“顶”上来，并且保证数据尽量不丢失(主从复制是最终一致性)。

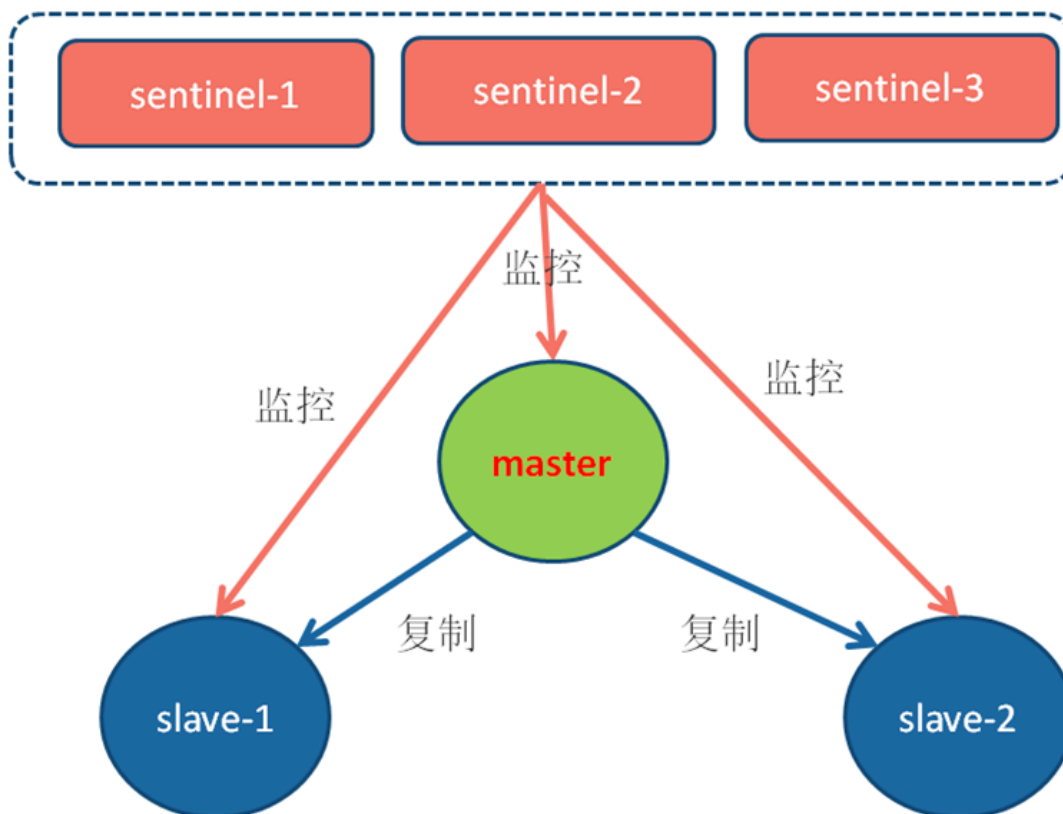
第二，从节点可以扩展主节点的读能力，一旦主节点不能支撑住大并发量的读操作，从节点可以在一定程度上帮助主节点分担读压力。

但是主从复制也带来了以下问题:

- 1、一旦主节点出现故障，需要手动将一个从节点晋升为主节点，同时需要修改应用方的主节点地址，还需要命令其他从节点去复制新的主节点,整个过程都需要人工干预。
- 2、主节点的写能力受到单机的限制。
- 3、主节点的存储能力受到单机的限制。

Redis Sentinel

Redis Sentinel是一个分布式架构，其中包含若干个Sentinel节点和Redis数据节点，每个Sentinel节点会对数据节点和其余Sentinel节点进行监控，当它发现节点不可达时，会对节点做下线标识。如果被标识的是主节点，它还会和其他Sentinel节点进行“协商”，当大多数Sentinel节点都认为主节点不可达时，它们会选举出一个Sentinel节点来完成自动故障转移的工作，同时会将这个变化实时通知给Redis应用方。整个过程完全是自动的，不需要人工来介入，所以这套方案很有效地解决了Redis的高可用问题。



Redis Sentinel的搭建

我们以3个 Sentinel节点、1个主节点、2个从节点组成一个Redis Sentinel进行说明。

启动主从的方式和普通的主从没有不同。

启动Sentinel节点

Sentinel节点的启动方法有两种:

方法一,使用redis-sentinel命令:

```
./redis-sentinel ../conf/reids.conf
```

方法二, 使用redis-server命令加--sentinel参数:

```
./redis-server ../conf/reids.conf --sentinel
```

两种方法本质上是一样的。

确认

Sentinel节点本质上是一个特殊的Redis节点, 所以也可以通过info命令来查询它的相关信息

```
[redis@iZwz9j203ithc4guluvb2wZ src]$ ./redis-cli -p 26885 info Sentinel
# Sentinel
sentinel_masters:1
sentinel_tilt:0
sentinel_running_scripts:0
sentinel_scripts_queue_length:0
sentinel_simulate_failure_flags:0
master0:name=mymaster,status=ok,address=127.0.0.1:6885,slaves=2,sentinels=1
```

实现原理

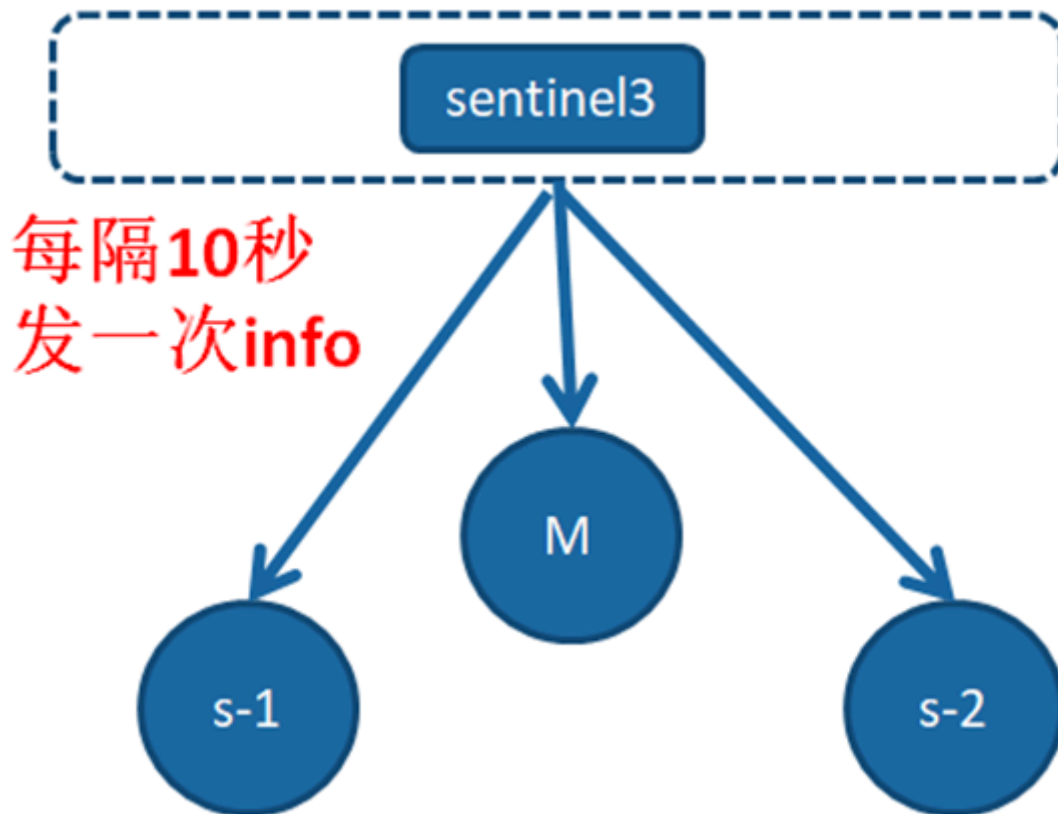
Redis Sentinel的基本实现中包含以下：

Redis Sentinel 的定时任务、主观下线和客观下线、Sentinel领导者选举、故障转移等等知识点，学习这些可以让我们对Redis Sentinel的高可用特性有更加深入的理解和认识。

三个定时监控任务

一套合理的监控机制是Sentinel节点判定节点不可达的重要保证，Redis Sentinel通过三个定时监控任务完成对各个节点发现和监控：

1、每隔10秒的定时监控

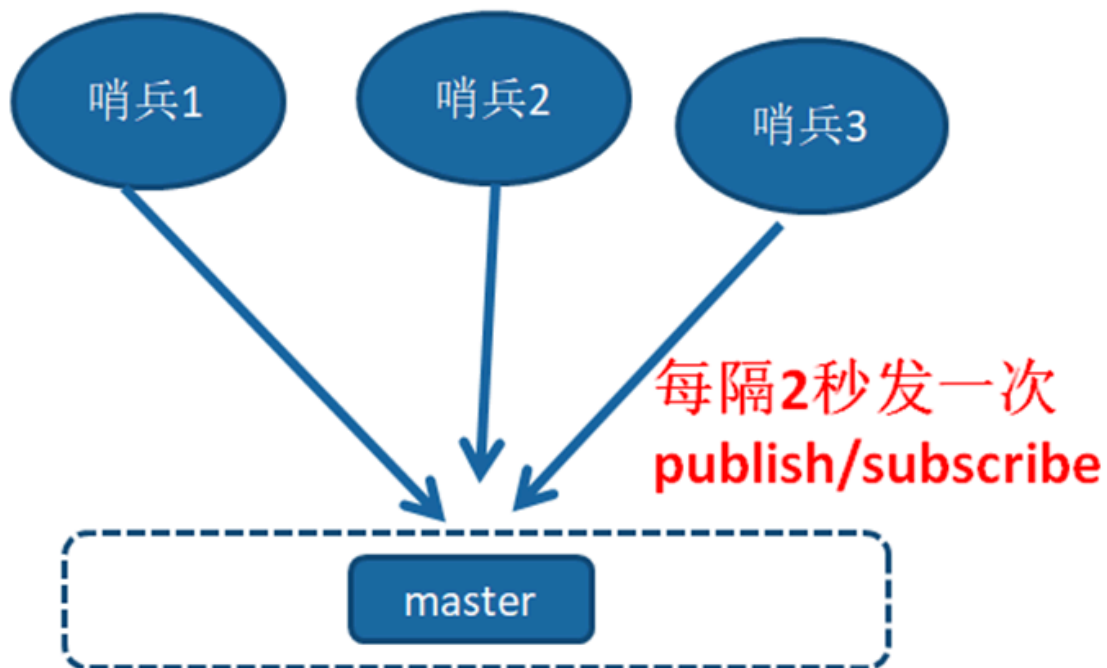


每隔10秒，每个Sentinel节点会向主节点和从节点发送info命令获取最新的拓扑结构，Sentinel节点通过对上述结果进行解析就可以找到相应的从节点。

这个定时任务的作用具体可以表现在三个方面：

- 1、通过向主节点执行info命令，获取从节点的信息,这也是为什么Sentinel节点不需要显式配置监控从节点。
- 2、当有新的从节点加入时都可以立刻感知出来。
- 3、节点不可达或者故障转移后，可以通过info命令实时更新节点拓扑信息。

2、每隔2秒的定时监控

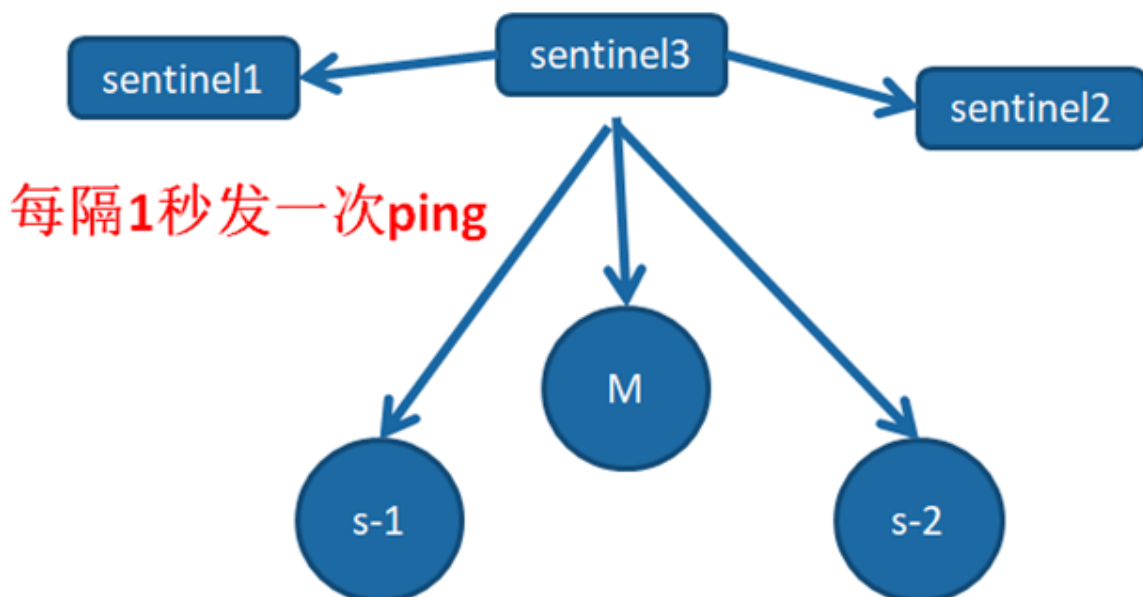


每隔2秒,每个Sentinel节点会向Redis数据节点的`sentinel:hello`频道上发送该Sentinel节点对于主节点的判断以及当前Sentinel节点的信息,同时每个Sentinel节点也会订阅该频道,来了解其他Sentinel节点以及它们对主节点的判断,所以这个定时任务可以完成以下两个工作:

发现新的Sentinel节点:通过订阅主节点的`sentinel:hello`了解其他的Sentinel节点信息,如果是新加入的Sentinel节点,将该Sentinel节点信息保存起来,并与该Sentinel节点创建连接。

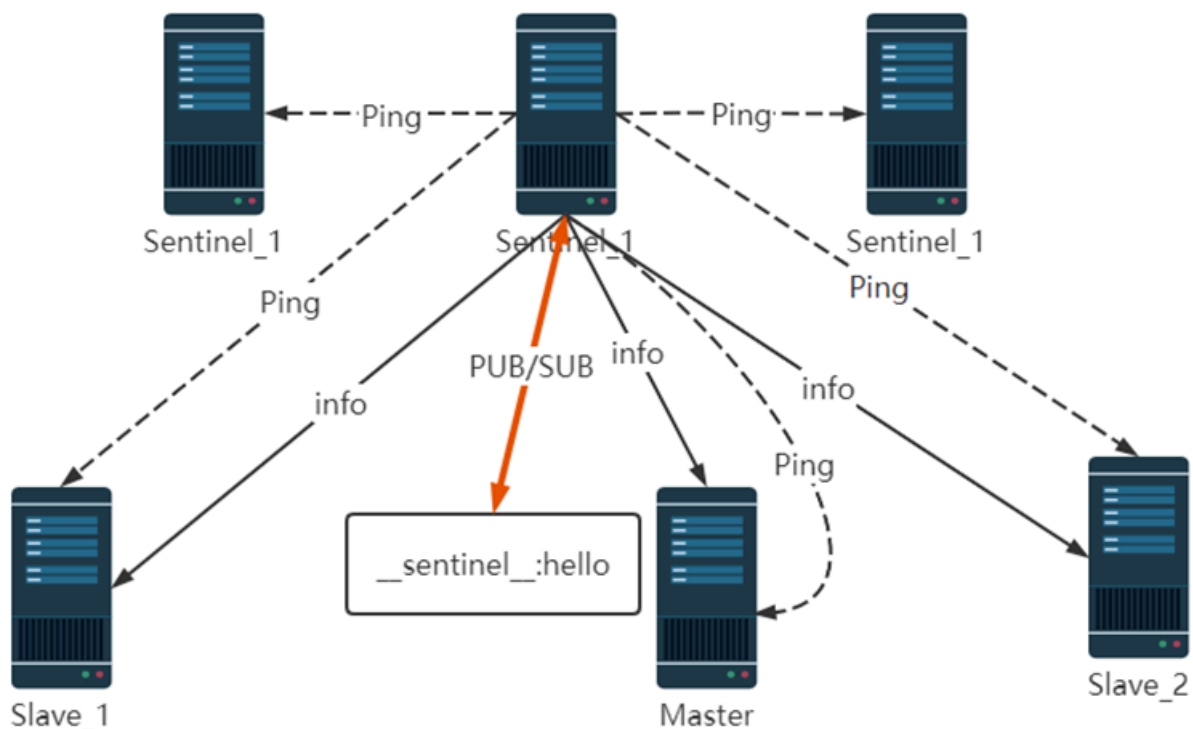
Sentinel节点之间交换主节点的状态,作为后面客观下线以及领导者选举的依据。

3、每隔1秒的定时监控



每隔1秒,每个Sentinel节点会向主节点、从节点、其余Sentinel节点发送一条ping命令做一次心跳检测,来确认这些节点当前是否可达。

通过上面的定时任务, Sentinel节点对主节点、从节点、其余Sentinel节点都建立起连接,实现了对每个节点的监控,这个定时任务是节点失败判定的重要依据。

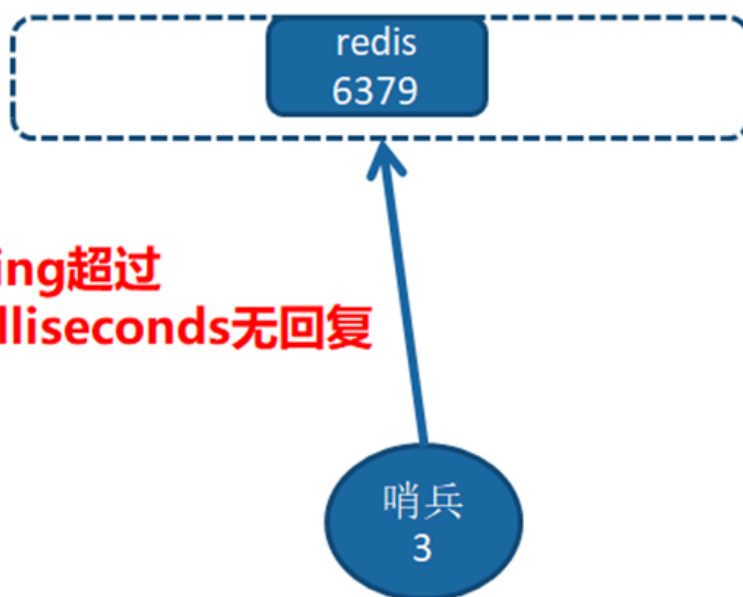


主观下线和客观下线

主观下线

主观下线

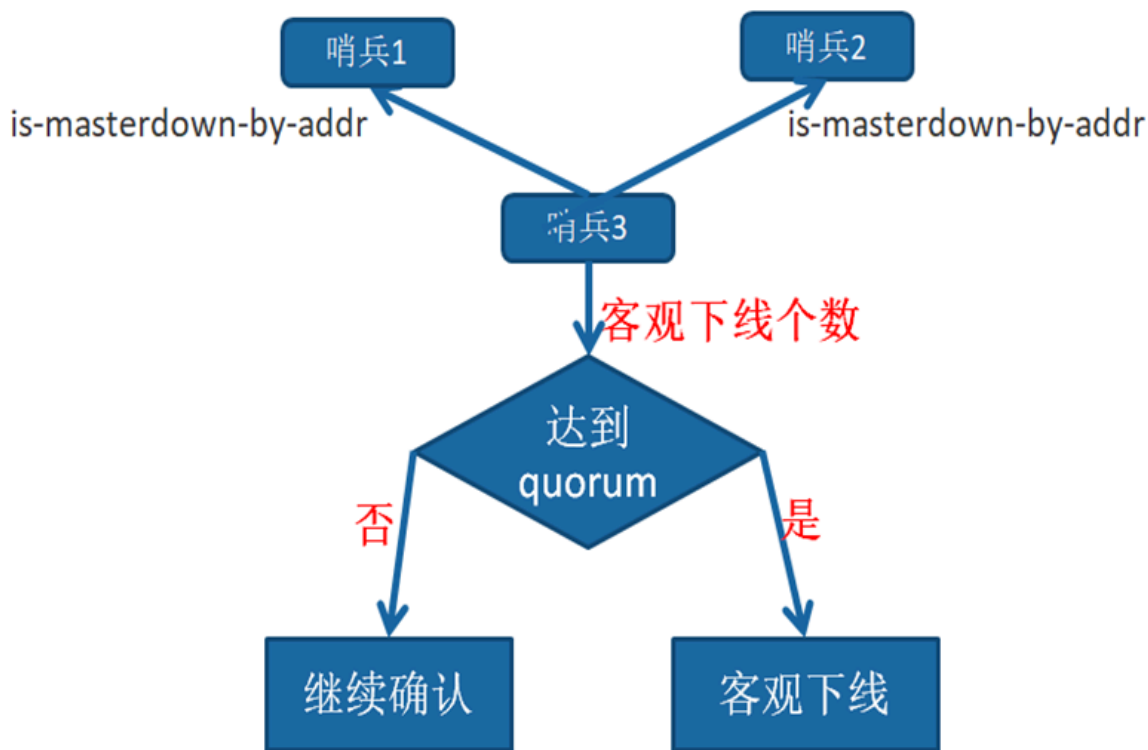
每隔1秒发一次ping超过
down-after-milliseconds无回复



```
sentinel myid e97b0b4c35e07d53cf070f76fe4d61147205a98b
sentinel deny-scripts-reconfig yes
sentinel monitor mymaster 127.0.0.1 6381 2
sentinel down-after-milliseconds mymaster 3000
```

上一小节介绍的第三个定时任务，每个Sentinel节点会每隔1秒对主节点、从节点、其他Sentinel节点发送ping命令做心跳检测,当这些节点超过down-after-milliseconds没有进行有效回复，Sentinel节点就会对该节点做失败判定，这个行为叫做主观下线。从字面意思也可以很容易看出主观下线是当前Sentinel节点的一家之言,存在误判的可能。

客观下线

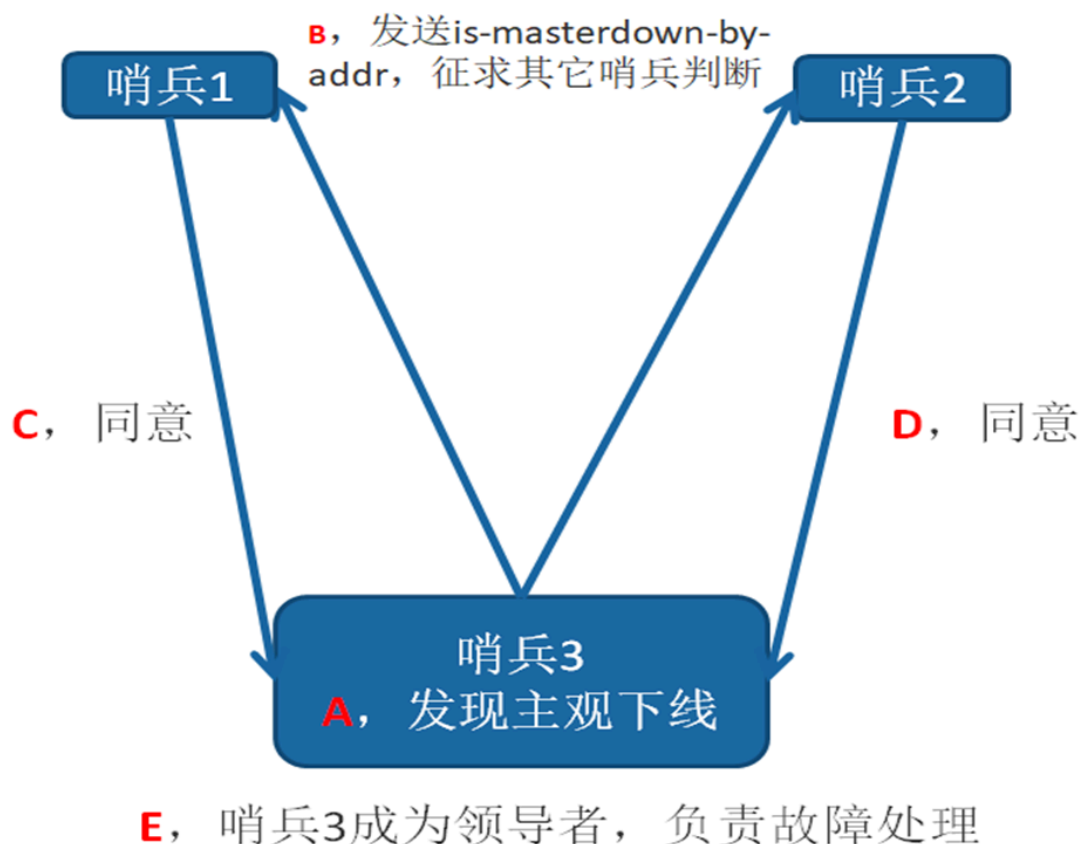


当Sentinel主观下线的节点是主节点时，该Sentinel节点会通过sentinel is-master-down-by-addr命令向其他Sentinel节点询问对主节点的判断，当超过<quorum>个数,Sentinel节点认为主节点确实有问题，这时该Sentinel节点会做出客观下线的决定，这样客观下线的含义是比较明显了，也就是大部分Sentinel节点都对主节点的下线做了同意的判定，那么这个判定就是客观的。

```
sentinel myid e97b0b4c35e07d53cf070f76fe4d61147205a98b
sentinel deny-scripts-reconfig yes
sentinel monitor mymaster 127.0.0.1 6381 2
sentinel down-after-milliseconds mymaster 3000
```

```
# MGR Master A is down
```


领导者Sentinel节点选举



假如Sentinel节点对于主节点已经做了客观下线, 那么是不是就可以立即进行故障转移了? 当然不是, 实际上故障转移的工作只需要一个Sentinel节点来完成即可, 所以 Sentinel节点之间会做一个领导者选举的工作, 选出一个Sentinel节点作为领导者进行故障转移的工作。Redis使用了Raft算法实现领导者选举, Redis Sentinel进行领导者选举的大致思路如下:

- 1) 每个在线的Sentinel节点都有资格成为领导者, 当它确认主节点主观下线时候, 会向其他Sentinel节点发送sentinel is-master-down-by-addr命令, 要求将自己设置为领导者。
- 2) 收到命令的Sentinel节点, 如果没有同意过其他Sentinel节点的sentinel is-master-down-by-addr命令, 将同意该请求, 否则拒绝。
- 3) 如果该Sentinel节点发现自己的票数已经大于等于 $\max(\text{quorum}, \text{num}(\text{sentinels})/2+1)$, 那么它将成为领导者。
- 4) 如果此过程没有选举出领导者, 将进入下一次选举。

选举的过程非常快, 基本上谁先完成客观下线, 谁就是领导者。

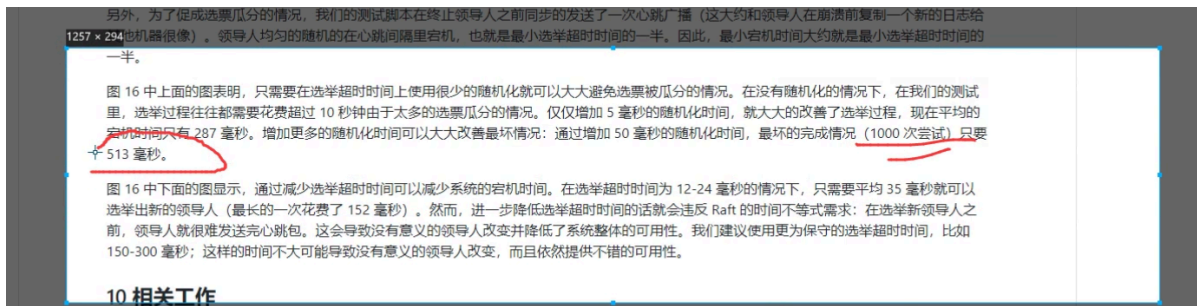
Raft协议的详细版本:

[raft-zh cn/raft-zh cn.md at master · maemual/raft-zh cn · GitHub](#)

如果你想手写一个Raft协议, 可以看下蚂蚁金服的开发生产的raft算法组件

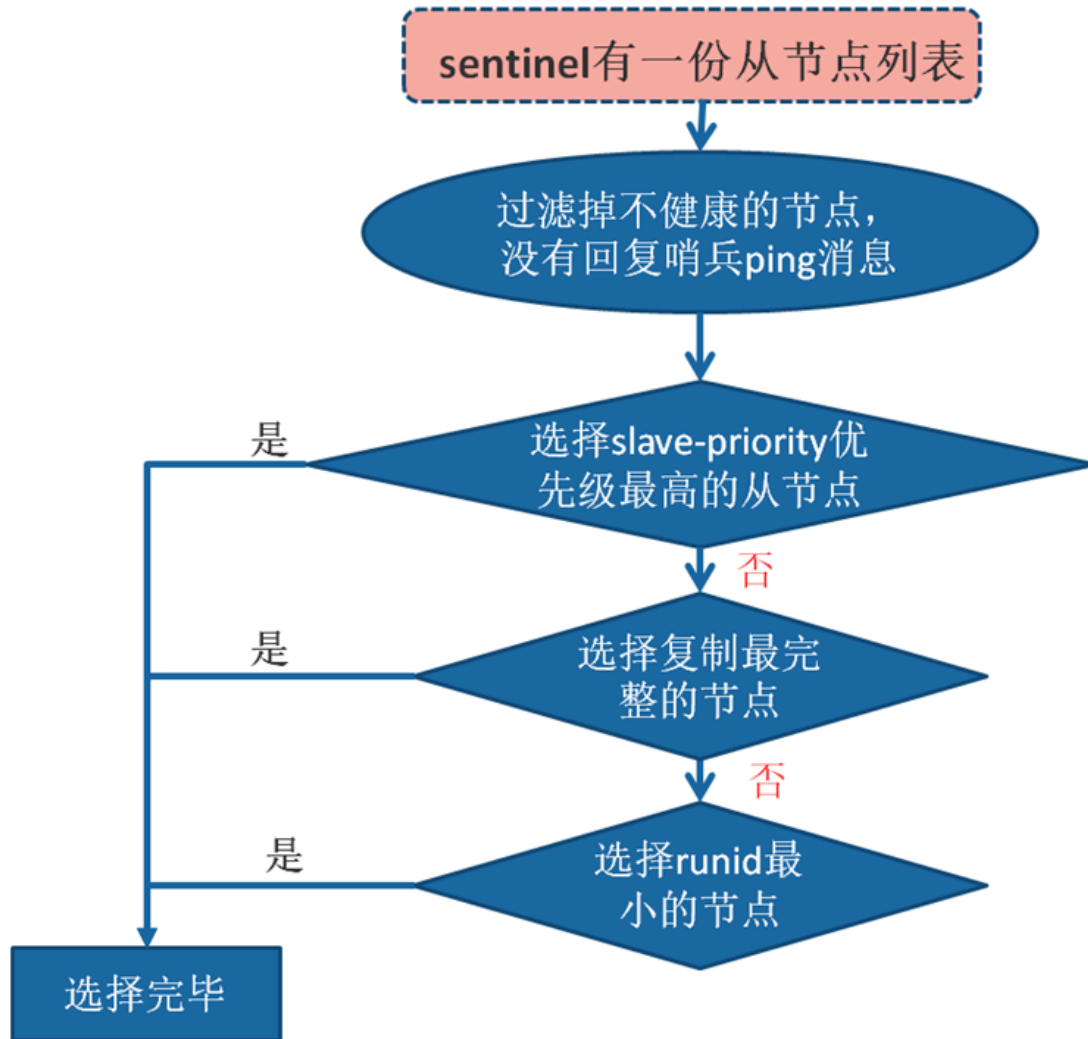
[GitHub - sofastack/sofa-jraft: A production-grade java implementation of RAFT consensus algorithm.](#)

选举很快的!!



故障转移

领导者选举出的Sentinel节点负责故障转移，具体步骤如下：



1)在从节点列表中选出一个节点作为新的主节点,选择方法如下:

a)过滤:“不健康”(主观下线、断线)、5秒内没有回复过Sentinel节点 ping响应、与主节点失联超过down-after-milliseconds*10秒。

b)选择slave-priority(从节点优先级)最高的从节点列表，如果存在则返回,不存在则继续。

c) 选择复制偏移量最大的从节点(复制的最完整)，如果存在则返回,不存在则继续。

d) 选择runid最小的从节点。

- 2) Sentinel领导者节点会对第一步选出来的从节点执行slaveof no one命令让其成为主节点。
- 3) Sentinel领导者节点会向剩余的从节点发送命令, 让它们成为新主节点的从节点,复制规则和parallel-syncs参数有关。
- 4) Sentinel节点集合会将原来的主节点更新为从节点, 并保持着对其关注, 当其恢复后命令它去复制新的主节点。

5、讲一讲Redis缓存的集群原理

以下笔记来自Redis体系课, 面试突击课只是抽取其中一些快速讲解:

集群原理

节点通信

通信流程

在分布式存储中需要提供维护节点元数据信息的机制,所谓元数据是指:节点负责哪些数据,是否出现故障等状态信息。常见的元数据维护方式分为:集中式和P2P方式。Redis集群采用P2P的Gossip (流言)协议, Gossip协议工作原理就是节点彼此不断通信交换信息,一段时间后所有的节点都会知道集群完整的信息,这种方式类似流言传播。

通信过程说明:

- 1)集群中的每个节点都会单独开辟一个TCP通道,用于节点之间彼此通信,通信端口号在基础端口上加10000。
- 2)每个节点在固定周期内通过特定规则选择几个节点发送ping消息。
- 3) 接收到ping消息的节点用pong消息作为响应。

集群中每个节点通过一定规则挑选要通信的节点, 每个节点可能知道全部节点,也可能仅知道部分节点,只要这些节点彼此可以正常通信, 最终它们会达到一致的状态。当节点出故障、新节点加入、主从角色变化、槽信息变更等事件发生时, 通过不断的ping/pong消息通信, 经过一段时间后所有的节点都会知道整个集群全部节点的最新状态, 从而达到集群状态同步的目的。

Gossip 消息

Gossip协议的主要职责就是信息交换。信息交换的载体就是节点彼此发送的Gossip消息, 了解这些消息有助于我们理解集群如何完成信息交换。

常用的Gossip消息可分为:ping消息、pong消息、meet消息、fail消息等,

meet消息:

用于通知新节点加入。消息发送者通知接收者加入到当前集群, meet消息通信正常完成后, 接收节点会加入到集群中并进行周期性的ping、pong消息交换。

ping消息:

集群内交换最频繁的消息, 集群内每个节点每秒向多个其他节点发送ping消息,用于检测节点是否在线和交换彼此状态信息。ping消息发送封装了自身节点和部分其他节点的状态数据。

pong消息:

当接收到ping、meet消息时, 作为响应消息回复给发送方确认消息正常通信。pong消息内部封装了自身状态数据。节点也可以向集群内广播自身的pong消息来通知整个集群对自身状态进行更新。

fail消息:

当节点判定集群内另一个节点下线时，会向集群内广播一个fail消息,其他节点接收到fail消息之后把对应节点更新为下线状态。

所有的消息格式划分为:消息头和消息体。消息头包含发送节点自身状态数据，接收节点根据消息头就可以获取到发送节点的相关数据。

集群内所有的消息都采用相同的消息头结构clusterMsg，它包含了发送节点关键信息，如节点id、槽映射、节点标识(主从角色，是否下线)等。消息体在Redis内部采用clusterMsg Data 结构声明。

消息体clusterMsgData定义发送消息的数据,其中ping,meet、pong都采用clusterMsgDataGossip数组作为消息体数据，实际消息类型使用消息头的type属性区分。每个消息体包含该节点的多个clusterMsgDataGossip结构数据，用于信息交换。

当接收到ping、meet消息时,接收节点会解析消息内容并根据自身的识别情况做出相应处理。

节点选择

虽然Gossip协议的信息交换机制具有天然的分布式特性，但它是有成本的。由于内部需要频繁地进行节点信息交换，而ping/pong消息会携带当前节点和部分其他节点的状态数据，势必会加重带宽和计算的负担。Redis集群内节点通信采用固定频率(定时任务每秒执行10次)。

因此节点每次选择需要通信的节点列表变得非常重要。通信节点选择过多虽然可以做到信息及时交换但成本过高。节点选择过少会降低集群内所有节点彼此信息交换频率，从而影响故障判定、新节点发现等需求的速度。因此Redis集群的Gossip协议需要兼顾信息交换实时性和成本开销。

消息交换的成本主要体现在单位时间选择发送消息的节点数量和每个消息携带的数据量。

1.选择发送消息的节点数量

集群内每个节点维护定时任务默认间隔1秒，每秒执行10次，定时任务里每秒随机选取5个节点，找出最久没有通信的节点发送ping消息，用于保证 Gossip信息交换的随机性。同时每100毫秒都会扫描本地节点列表，如果发现节点最近一次接受pong消息的时间大于cluster_node_timeout/2，则立刻发送ping消息，防止该节点信息太长时间未更新。

根据以上规则得出每个节点每秒需要发送ping消息的数量= 1 +10

- num(node.pong_received > cluster_node_timeout/2)，因此cluster_node_timeout参数对消息发送的节点数量影响非常大。当我们的带宽资源紧张时，可以适当调大这个参数，如从默认15秒改为30秒来降低带宽占用率。过度调大cluster_node_timeout 会影响消息交换的频率从而影响故障转移、槽信息更新、新节点发现的速度。因此需要根据业务容忍度和资源消耗进行平衡。同时整个集群消息总交换量也跟节点数成正比。

2 消息数据量

每个ping消息的数据量体现在消息头和消息体中,其中消息头主要占用空间的字段是myslots [CLUSTER_SLOTS/8]，占用2KB，这块空间占用相对固定。消息体会携带一定数量的其他节点信息用于信息交换。

根消息体携带数据量跟集群的节点数息息相关，更大的集群每次消息通信的成本也就更高，因此对于Redis集群来说并不是大而全的集群更好。

故障转移

Redis集群自身实现了高可用。高可用首先需要解决集群部分失败的场景：当集群内少量节点出现故障时通过自动故障转移保证集群可以正常对外提供服务。

故障发现

当集群内某个节点出现问题时，需要通过一种健壮的方式保证识别出节点是否发生了故障。Redis集群内节点通过ping/pong消息实现节点通信，消息不但可以传播节点槽信息，还可以传播其他状态如:主从状态、节点故障等。因此故障发现也是通过消息传播机制实现的,主要环节包括:主观下线(pfail)和客观下线(fail)。

主观下线:

指某个节点认为另一个节点不可用，即下线状态，这个状态并不是最终的故障判定,只能代表一个节点的意见,可能存在误判情况。

客观下线:

指标记一个节点真正的下线，集群内多个节点都认为该节点不可用,从而达成共识的结果。如果是持有槽的主节点故障，需要为该节点进行故障转移。

主观下线

集群中每个节点都会定期向其他节点发送ping消息，接收节点回复pong消息作为响应。如果在cluster-node-timeout时间内通信一直失败,则发送节点会认为接收节点存在故障，把接收节点标记为主观下线(pfail)状态。

流程说明:

- 1) 节点a发送ping消息给节点b，如果通信正常将接收到pong消息，节点 a更新最近一次与节点b的通信时间。
- 2) 如果节点 a与节点b通信出现问题则断开连接，下次会进行重连。如果一直通信失败,则节点a记录的与节点b最后通信时间将无法更新。
- 3) 节点a内的定时任务检测到与节点b最后通信时间超高cluster-node-timeout时，更新本地对节点b的状态为主观下线(pfail)。

主观下线简单来讲就是，当cluster-note-timeout时间内某节点无法与另一个节点顺利完成ping消息通信时，则将该节点标记为主观下线状态。每个节点内的clusterstate结构都需要保存其他节点信息,用于从自身视角判断其他节点的状态。

Redis集群对于节点最终是否故障判断非常严谨，只有一个节点认为主观下线并不能准确判断是否故障。

比如节点6379与6385通信中断，导致6379判断6385为主观下线状态，但是6380与6385节点之间通信正常,这种情况不能判定节点6385发生故障。因此对于一个健壮的故障发现机制,需要集群内大多数节点都判断6385故障时,才能认为6385确实发生故障,然后为6385节点进行故障转移。而这种多个节点协作完成故障发现的过程叫做客观下线。

客观下线

当某个节点判断另一个节点主观下线后，相应的节点状态会跟随消息在集群内传播。

ping/pong消息的消息体会携带集群1/10的其他节点状态数据，当接受节点发现消息体中含有主观下线的节点状态时，会在本地找到故障节点的ClusterNode结构，保存到下线报告链表中。

通过Gossip消息传播，集群内节点不断收集到故障节点的下线报告。当半数以上持有槽的主节点都标记某个节点是主观下线时。触发客观下线流程。这里有两个问题:

- 1)为什么必须是负责槽的主节点参与故障发现决策?因为集群模式下只有处理槽的主节点才负责读写请求和集群槽等关键信息维护，而从节点只进行主节点数据和状态信息的复制。

2) 为什么半数以上处理槽的主节点? 必须半数以上是为了应对网络分区等原因造成的集群分割情况, 被分割的小集群因为无法完成从主观下线到客观下线这一关键过程, 从而防止小集群完成故障转移之后继续对外提供服务。

尝试客观下线

集群中的节点每次接收到其他节点的pfail状态, 都会尝试触发客观下线,

流程说明:

- 1) 首先统计有效的下线报告数量, 如果小于集群内持有槽的主节点总数的一半则退出。
- 2) 当下线报告大于槽主节点数量一半时, 标记对应故障节点为客观下线状态。
- 3) 向集群广播一条fail消息, 通知所有的节点将故障节点标记为客观下线, fail消息的消息体只包含故障节点的ID。

广播fail消息是客观下线的最后一步, 它承担着非常重要的职责:

通知集群内所有的节点标记故障节点为客观下线状态并立刻生效。

通知故障节点的从节点触发故障转移流程。

故障恢复

故障节点变为客观下线后, 如果下线节点是持有槽的主节点则需要从它的从节点中选出一个替换它, 从而保证集群的高可用。下线主节点的所有从节点承担故障恢复的义务, 当从节点通过内部定时任务发现自身复制的主节点进入客观下线时, 将会触发故障恢复流程。

资格检查

每个从节点都要检查最后与主节点断线时间, 判断是否有资格替换故障的主节点。如果从节点与主节点断线时间超过 $\text{cluster-node-time} * \text{cluster-slave-validity-factor}$, 则当前从节点不具备故障转移资格。参数 $\text{cluster-slave-validity-factor}$ 用于从节点的有效因子, 默认为10。

准备选举时间

当从节点符合故障转移资格后, 更新触发故障选举的时间, 只有到达该时间后才能执行后续流程。

这里之所以采用延迟触发机制, 主要是通过对多个从节点使用不同的延迟选举时间来支持优先级问题。复制偏移量越大说明从节点延迟越低, 那么它应该具有更高的优先级来替换故障主节点。

所有的从节点中复制偏移量最大的将提前触发故障选举流程。

主节点b进入客观下线后, 它的三个从节点根据自身复制偏移量设置延迟选举时间, 如复制偏移量最大的节点slave b-1延迟1秒执行, 保证复制延迟低的从节点优先发起选举。

发起选举

当从节点定时任务检测到故障选举时间($\text{failover_auth_time}$) 到达后, 发起选举流程如下:

(1) 更新配置纪元

配置纪元是一个只增不减的整数, 每个主节点自身维护一个配置纪元($\text{clusterNode.configEpoch}$)标示当前主节点版本, 所有主节点的配置纪元都不相等, 从节点会复制主节点的配置纪元。整个集群又维护一个全局的配置纪元($\text{clusterstate.currentEpoch}$), 用于记录集群内所有主节点配置纪元的最大版本。执行 `cluster info` 命令可以查看配置纪元信息:

```
127.0.0.1:6900> cluster info
cluster_state:ok
cluster_slots_assigned:16384
cluster_slots_ok:16384
cluster_slots_pfail:0
cluster_slots_fail:0
cluster_known_nodes:6
cluster_size:3
cluster_current_epoch:8
cluster_my_epoch:6
```

配置纪元的主要作用:

标示集群内每个主节点的不同版本和当前集群最大的版本。

每次集群发生重要事件时, 这里的重要事件指出现新的主节点(新加入的或者由从节点转换而来), 从节点竞争选举。都会递增集群全局的配置纪元并赋值给相关主节点,用于记录这一关键事件。

主节点具有更大的配置纪元代表了更新的集群状态, 因此当节点间进行ping/pong消息交换时, 如出现slots等关键信息不一致时, 以配置纪元更大的一方为准, 防止过时的消息状态污染集群。

配置纪元的应用场景有:

新节点加入。槽节点映射冲突检测。从节点投票选举冲突检测。

选举投票

只有持有槽的主节点才会处理故障选举消息(FAILOVER_AUTH_REQUEST), 因为每个持有槽的节点在一个配置纪元内都有唯一的一张选票, 当接到第一个请求投票的从节点消息时回复FAILOVER_AUTH_ACK消息作为投票, 之后相同配置纪元内其他从节点的选举消息将忽略。

投票过程其实是一个领导者选举的过程, 如集群内有N个持有槽的主节点代表有N张选票。由于在每个配置纪元内持有槽的主节点只能投票给一个从节点, 因此只能有一个从节点获得 $N/2+1$ 的选票, 保证能够找出唯一的从节点。

Redis集群没有直接使用从节点进行领导者选举, 主要因为从节点数必须大于等于3个才能保证凑够 $N/2+1$ 个节点, 将导致从节点资源浪费。使用集群内所有持有槽的主节点进行领导者选举, 即使只有一个从节点也可以完成选举过程。

当从节点收集到 $N/2+1$ 个持有槽的主节点投票时, 从节点可以执行替换主节点操作, 例如集群内有5个持有槽的主节点, 主节点b故障后还有4个, 当其中一个从节点收集到3张投票时代表获得了足够的选票可以进行替换主节点操作。

投票作废:每个配置纪元代表了一次选举周期, 如果在开始投票之后的cluster-node-timeout*2时间内从节点没有获取足够数量的投票, 则本次选举作废。从节点对配置纪元自增并发起下一轮投票, 直到选举成功为止。

替换主节点

当从节点收集到足够的选票之后,触发替换主节点操作:

- 1)当前从节点取消复制变为主节点。
- 2)执行clusterDelslot 操作撤销故障主节点负责的槽, 并执行clusterAddSlot把这些槽委派给自己。
- 3)向集群广播自己的pong消息, 通知集群内所有的节点当前从节点变为主节点并接管了故障主节点的槽信息。

故障转移时间

在介绍完故障发现和恢复的流程后,这时我们可以估算出故障转移时间:

- 1) 主观下线(pfail) 识别时间=cluster-node-timeout。
- 2) 主观下线状态消息传播时间 $\leq$ cluster-node-timeout/2。消息通信机制对超过cluster-node-timeout/2未通信节点会发起ping消息, 消息体在选择包含哪些节点时会优先选取下线状态节点, 所以通常这段时间内能够收集到半数以上主节点的pfail 报告从而完成故障发现。
- 3)从节点转移时间 $\leq$ 1000毫秒。由于存在延迟发起选举机制,偏移量最大的从节点会最多延迟1秒发起选举。通常第一次选举就会成功, 所以从节点执行转移时间在1秒以内。

根据以上分析可以预估出故障转移时间, 如下:

$\text{failover-time(毫秒)} \leq \text{cluster-node-timeout}$

- $\text{cluster-node-timeout}/2 + 1000$

因此, 故障转移时间跟cluster-node-timeout参数息息相关, 默认15秒。配置时可以根据业务容忍度做出适当调整, 但不是越小越好。

6、讲一讲Redis分布式锁的原理

以下笔记来自Redis体系课, 面试突击课只是抽取其中一些快速讲解:

Redis分布式锁最简单的实现

想要实现分布式锁, 必须要求 Redis 有「互斥」的能力, 我们可以使用 SETNX 命令, 这个命令表示SET if Not Exists, 即如果 key 不存在, 才会设置它的值, 否则什么也不做。

两个客户端进程可以执行这个命令, 达到互斥, 就可以实现一个分布式锁。

客户端 1 申请加锁, 加锁成功:

客户端 2 申请加锁, 因为它后到达, 加锁失败:

redis-cli.exe

```
127.0.0.1:6379> setnx lock 1
(integer) 1
127.0.0.1:6379>
```

redis-cli.exe

```
127.0.0.1:6379> setnx lock 1
(integer) 0
127.0.0.1:6379> _
```

此时，加锁成功的客户端，就可以去操作「共享资源」，例如，修改 MySQL 的某一行数据，或者调用一个 API 请求。

操作完成后，还要及时释放锁，给后来者让出操作共享资源的机会。如何释放锁呢？

也很简单，直接使用 DEL 命令删除这个 key 即可，这个逻辑非常简单。

redis-cli.exe

```
127.0.0.1:6379> setnx lock 1
(integer) 1
127.0.0.1:6379> del lock
(integer) 1
127.0.0.1:6379>
```

redis-cli.exe

```
127.0.0.1:6379> setnx lock 1
(integer) 0
127.0.0.1:6379> setnx lock 1
(integer) 1
127.0.0.1:6379> _
```

但是，它存在一个很大的问题，当客户端 1 拿到锁后，如果发生下面的场景，就会造成「死锁」：

- 1、程序处理业务逻辑异常，没及时释放锁
- 2、进程挂了，没机会释放锁

这时，这个客户端就会一直占用这个锁，而其它客户端就「永远」拿不到这把锁了。怎么解决这个问题呢？

如何避免死锁？

我们很容易想到的方案是，在申请锁时，给这把锁设置一个「租期」。

在 Redis 中实现时，就是给这个 key 设置一个「过期时间」。这里我们假设，操作共享资源的时间不会超过 10s，那么在加锁时，给这个 key 设置 10s 过期即可：

```
SETNX lock 1    // 加锁
EXPIRE lock 10  // 10s后自动过期
```

```
(integer) 1
127.0.0.1:6379> setnx lock 1
(integer) 0
127.0.0.1:6379> expire lock 10
(integer) 1
127.0.0.1:6379>

redis-cli.exe
127.0.0.1:6379> setnx lock 1
(integer) 0
127.0.0.1:6379> setnx lock 1
(integer) 1
127.0.0.1:6379> setnx lock 1
(integer) 0
127.0.0.1:6379> setnx lock 1
(integer) 1
127.0.0.1:6379>
```

这样一来，无论客户端是否异常，这个锁都可以在 10s 后被「自动释放」，其它客户端依旧可以拿到锁。

但现在还是有问题：

现在的操作，加锁、设置过期是 2 条命令，有没有可能只执行了第一条，第二条却「来不及」执行的情况发生呢？例如：

- SETNX 执行成功，执行EXPIRE 时由于网络问题，执行失败
- SETNX 执行成功，Redis 异常宕机，EXPIRE 没有机会执行
- SETNX 执行成功，客户端异常崩溃，EXPIRE也没有机会执行

总之，这两条命令不能保证是原子操作（一起成功），就有潜在的风险导致过期时间设置失败，依旧发生「死锁」问题。

在 Redis 2.6.12 之后，Redis 扩展了 SET 命令的参数，用这一条命令就可以了：

```
SET lock 1 EX 10 NX
```

```
127.0.0.1:6379> set lock 1 ex 10 nx
OK
```

锁被别人释放怎么办？

上面的命令执行时，每个客户端在释放锁时，都是「无脑」操作，并没有检查这把锁是否还「归自己持有」，所以就会发生释放别人锁的风险，这样的解锁流程，很不「严谨」！如何解决这个问题呢？

解决办法是：客户端在加锁时，设置一个只有自己知道的「唯一标识」进去。

例如，可以是自己的线程 ID，也可以是一个 UUID（随机且唯一），这里我们以UUID 举例：

```
SET lock $uuid EX 20 NX
```

之后，在释放锁时，要先判断这把锁是否还归自己持有，伪代码可以这么写：

```
if redis.get("lock") == $uuid:
    redis.del("lock")
```

这里释放锁使用的是 GET + DEL 两条命令，这时，又会遇到我们前面讲的原子性问题了。这里可以使用 lua脚本来解决。

安全释放锁的 Lua 脚本如下：

```
if redis.call("GET",KEYS[1]) == ARGV[1]
then
    return redis.call("DEL",KEYS[1])
else
    return 0
end
```

好了，这样一路优化，整个的加锁、解锁的流程就更「严谨」了。

这里我们先小结一下，基于 Redis 实现的分布式锁，一个严谨的流程如下：

1、加锁

```
SET lock_key $unique_id EX $expire_time NX
```

2、操作共享资源

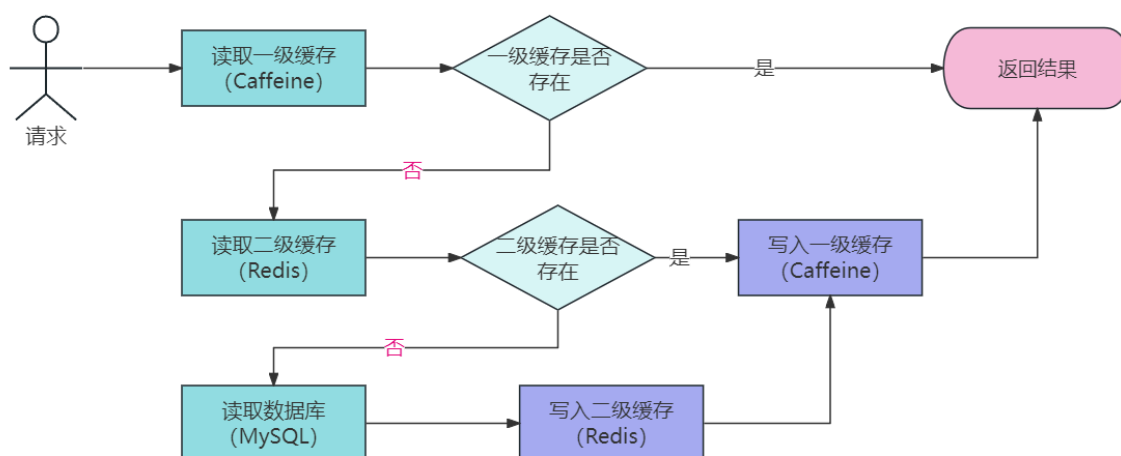
3、释放锁：Lua 脚本，先 GET 判断锁是否归自己，再DEL 释放锁

7、讲一讲Redis实现多级缓存

Caffeine+Redis两级缓存架构

在高性能的服务项目中，我们一般会将一些热点数据存储到 Redis这类缓存中间件中，只有当缓存的访问没有命中时再查询数据库。在提升访问速度的同时，也能降低数据库的压力。

但是在一些场景下单纯使用 Redis 的分布式缓存不能满足高性能的要求，所以还需要加入使用本地缓存 Caffeine，从而再次提升程序的响应速度与性能。于是，就产生了使用本地缓存（Caffeine）作为一级缓存，再加上分布式缓存（Redis）作为二级缓存的两级缓存架构。



两级缓存架构优缺点

优点：

- 一级缓存基于应用的内存，访问速度非常快，对于一些变更频率低、实时性要求低的数据，可以放在本地缓存中，提升访问速度；
- 使用一级缓存能够减少和 Redis 的二级缓存的远程数据交互，减少网络 I/O 开销，降低这一过程中在网络通信上的耗时。

缺点：

- 数据一致性问题：两级缓存与数据库的数据要保持一致，一旦数据发生了修改，在修改数据库的同时，一级缓存、二级缓存应该同步更新。
- 分布式多应用情况下：一级缓存之间也会存在一致性问题，当一个节点下的本地缓存修改后，需要通知其他节点也刷新本地一级缓存中的数据，否则会出现读取到过期数据的情况。
- 缓存的过期时间、过期策略以及多线程的问题

具体代码见：

<https://www.mashibing.com/study?courseNo=2598&sessionNo=105208&systemId=1&courseVersionId=3485>

课时14	清除、更新异步监听	2分24秒
章节2:Caffeine+Redis两级缓存架构 (7节)		
课时15	两级缓存架构介绍与优缺点	8分36秒
课时16	两级缓存实战-准备工作	6分55秒
课时17	两级缓存实战-版本1	6分5秒
课时18	两级缓存实战-版本1注意事项	2分1秒
课时19	注解方式两级缓存架构实战	11分57秒
课时20	自定义注解实现两级缓存架构实战	9分19秒
课时21	两级缓存架构的缓存一致性问题与实战解决	14分22秒

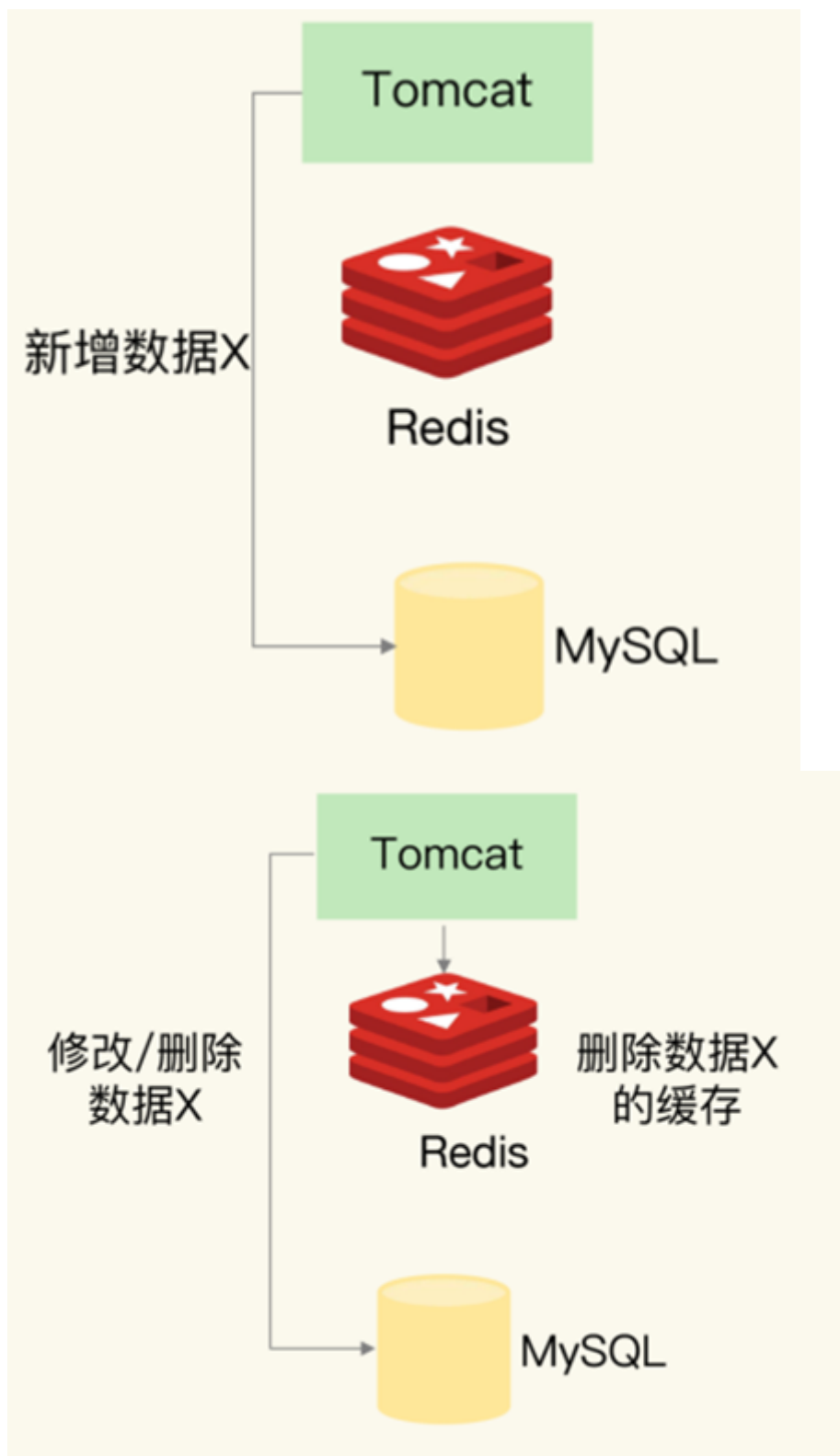
8、讲一讲Redis缓存的一致性问题

以下笔记来自Redis体系课，面试突击课只是抽取其中一些快速讲解：

数据一致性

只要使用到缓存，无论是本地内存做缓存还是使用 redis 做缓存，那么就会存在数据同步的问题。

我以 Tomcat 向 MySQL 中写入和删改数据为例，来给你解释一下，数据的增删改操作具体是如何进行的。



我们分析一下几种解决方案，

- 1、先更新缓存，再更新数据库
- 2、先更新数据库，再更新缓存
- 3、先删除缓存，后更新数据库
- 4、先更新数据库，后删除缓存

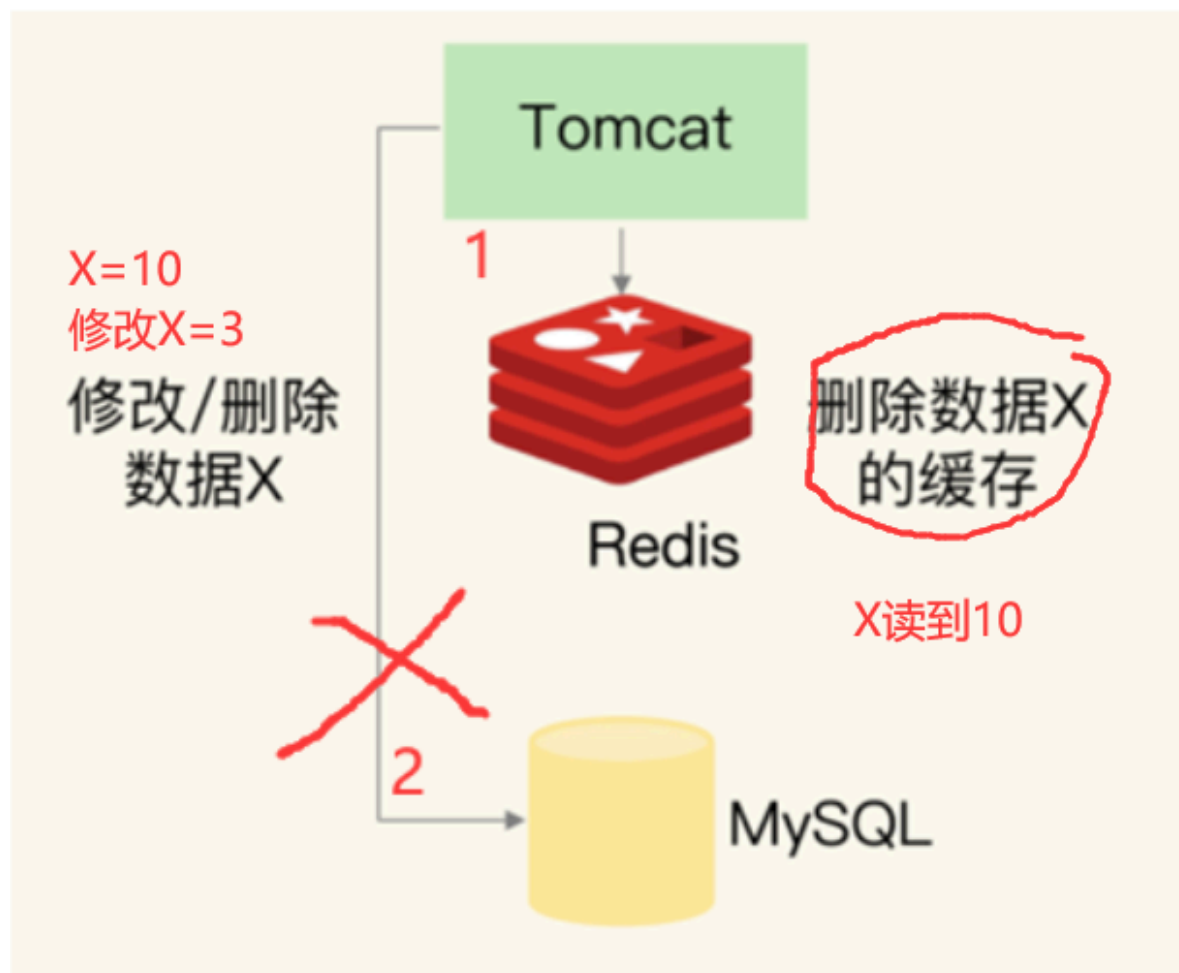
新增数据类

如果是新增数据，数据会直接写到数据库中，不用对缓存做任何操作，此时，缓存中本身就没有新增数据，而数据库中是最新值，此时，缓存和数据库的数据是一致的。

更新缓存类

1、先更新缓存，再更新DB

这个方案我们一般不考虑。原因是更新缓存成功，更新数据库出现异常了，导致缓存数据与数据库数据完全不一致，而且很难察觉，因为缓存中的数据一直都存在。



2、先更新DB，再更新缓存

这个方案也我们一般不考虑，原因跟第一个一样，数据库更新成功了，缓存更新失败，同样会出现数据不一致问题。同时还有以下问题

1) 并发问题:

同时有请求A和请求B进行更新操作，那么会出现

- (1) 线程A更新了数据库
- (2) 线程B更新了数据库
- (3) 线程B更新了缓存
- (4) 线程A更新了缓存

这就出现请求A更新缓存应该比请求B更新缓存早才对，但是因为网络等原因，B却比A更早更新了缓存。这就导致了脏数据，因此不考虑。

2) 业务场景问题

如果你是一个写数据库场景比较多，而读数据场景比较少的业务需求，采用这种方案就会导致，数据压根还没读到，缓存就被频繁的更新，浪费性能。

除了更新缓存之外，我们还有一种就是删除缓存。

到底是选择更新缓存还是淘汰缓存呢？

主要取决于“更新缓存的复杂度”，更新缓存的代价很小，此时我们应该更倾向于更新缓存，以保证更高的缓存命中率，更新缓存的代价很大，此时我们应该更倾向于淘汰缓存。

删除缓存类

3、先删除缓存，后更新DB

该方案也会出问题，具体出现的原因如下。

- 1、此时来了两个请求，请求 A（更新操作）和请求 B（查询操作）
- 2、请求 A 会先删除 Redis 中的数据，然后去数据库进行更新操作；
- 3、此时请求 B 看到 Redis 中的数据时空的，会去数据库中查询该值，补录到 Redis 中；
- 4、但是此时请求 A 并没有更新成功，或者事务还未提交，请求 B 去数据库查询得到旧值；
- 5、那么这时候就会产生数据库和 Redis 数据不一致的问题。

如何解决呢？其实最简单的解决办法就是延时双删的策略。就是

- (1) 先淘汰缓存
- (2) 再写数据库
- (3) 休眠1秒，再次淘汰缓存

这段伪代码就是“延迟双删”

```
redis.delkey(x)
db.update(x)
Thread.sleep(N)
redis.delkey(x)
```

这么做，可以将1秒内所造成的缓存脏数据，再次删除。

那么，这个1秒怎么确定的，具体该休眠多久呢？

针对上面的情形，读该自行评估自己的项目的读数据业务逻辑的耗时。然后写数据的休眠时间则在读数据业务逻辑的耗时基础上，加几百ms即可。这么做的目的，就是确保读请求结束，写请求可以删除读请求造成的缓存脏数据。

但是上述的保证事务提交完以后再进行删除缓存还有一个问题，就是如果你使用的是** Mysql 的读写分离的架构**的话，那么其实主从同步之间也会有时间差。

此时来了两个请求，请求 A（更新操作）和请求 B（查询操作）

请求 A 更新操作，删除了
Redis,

请求主库进行更新操作，主库与从库进行同步数据的操作，

请 B 查询操作，发现 Redis 中没有数据，

去从库中拿去数据，此时同步数据还未完成，拿到的数据是旧数据。

此时的解决办法有两个：

1、还是使用双删延时策略。只是，睡眠时间修改为在主从同步的延时时间基础上，加几百ms。

2、就是如果是对 Redis 进行填充数据的查询数据库操作，那么就强制将其指向主库进行查询。

继续深入，**采用这种同步淘汰策略，吞吐量降低怎么办？**

那就将第二次删除作为异步的。自己起一个线程，异步删除。这样，写的请求就不用沉睡一段时间了，再返回。这么做，加大吞吐量。

继续深入，**第二次删除,如果删除失败怎么办？**

所以，我们引出了，下面的第四种策略，先更新数据库，再删缓存。

4、先更新DB，后删除缓存

这种方式，被称为Cache Aside Pattern，读的时候，先读缓存，缓存没有的话，就读数据库，然后取出数据后放入缓存，同时返回响应。更新的时候，先更新数据库，然后再删除缓存。